

What Drives Data-Science Salaries? A Fully Bayesian Regression Analysis

Gamma vs. Log-Normal GLMs with a log link, both with covariate-dependent dispersion, estimated by MCMC in JAGS

Giannicola Mongelli, 2254343

Contents

1	Introduction	2
1.1	Research context and objectives	2
2	Illustration of the Dataset	2
2.1	Data loading and overview	2
2.2	Univariate exploration of the response	3
2.3	Categorical and discrete covariate summaries	4
2.4	Bivariate relationships	5
2.5	Response and predictor preparation	6
3	Models and Inferential Goals	8
3.1	The two competing models	8
3.2	Prior specification (elicited before seeing the data)	8
3.3	JAGS model files	8
3.4	Role of the parameters and inferential goals	10
3.5	Model validation: parameter recovery on simulated data	10
4	Model 1: Gamma GLM	13
4.1	Fit on the real data	13
4.2	Interpretation of the model	14
4.3	Diagnostics	15
4.4	Heteroscedastic vs constant dispersion: Gamma family	17
5	Model 2: Log-Normal GLM	17
5.1	Fit on the real data	18
5.2	Interpretation of the model	18
5.3	Diagnostics	20
5.4	Heteroscedastic vs constant dispersion: Log-Normal family	22
6	Model Comparison	22
7	Posterior Predictive Checks	23
8	Inference and Prediction	26
8.1	Prediction for new employee profiles	26
9	Comparative Analysis: Frequentist vs. Bayesian	28

1 Introduction

This project uses a Bayesian approach to study data-science salaries. The response is `salary_in_usd` (annual gross salary in US dollars), a positive and right-skewed variable. We fit two Bayesian GLMs by MCMC in JAGS and compare them with the Deviance Information Criterion (DIC). The main question is:

Which distribution, Gamma or Log-Normal, fits the salary data better, using the same covariates and a log link?

Both models share the same response, the same three covariates, and the same log link for the mean: a Gamma GLM, which targets the expected salary, and a Log-Normal GLM, which targets the median salary. Both models also let dispersion depend on the same covariates, the Gamma's shape and the Log-Normal's precision, using matching priors and the same number of extra parameters. This keeps the comparison in Section 6 fair: it tests sampling family, not flexibility. Each family is also fitted with constant dispersion as a simpler baseline, so we can check if the extra flexibility is worth it (Sections 4.4 and 5.4), and compare families at matched complexity (Section 6). The full notation for both models is given in Section 1.2.

Under the Bayesian paradigm the posterior of the parameters is

$$\pi(\theta \mid y) \propto L_y(\theta) \pi(\theta),$$

where $\pi(\theta)$ is the prior and $L_y(\theta)$ the likelihood.

1.1 Research context and objectives

Salaries depend on seniority, company size, location, remote work, and the year of the offer. We have two goals. First, compare the two families by DIC, at matched complexity, to see which fits salary better once covariates are in both the mean and the dispersion. Second, estimate and interpret the effects of interest: the multiplicative effect e^{β_j} of each covariate on expected salary, with full posterior uncertainty, and the sign and size of the dispersion effects γ_j and δ_j . We report point estimates, HPD credible intervals, a prior-vs-posterior comparison, convergence diagnostics for the key parameters, parameter recovery on simulated data, posterior predictive checks, and a comparison with frequentist inference, done the same way for both models.

2 Illustration of the Dataset

2.1 Data loading and overview

The data are the public Data Science Salaries dataset from `ai-jobs.net` (`salaries.ai-jobs.net`, also on Kaggle), an open survey of data-science compensation worldwide. Each row is one employee/year record.

```
salaries <- read.csv("ds_salaries.csv", stringsAsFactors = FALSE)
if (names(salaries)[1] %in% c("X", "")) salaries <- salaries[, -1]

cat("Dataset dimensions:", nrow(salaries), "observations x",
    ncol(salaries), "variables\n")
```

```
## Dataset dimensions: 607 observations x 11 variables
```

```
str(salaries)
```

```
## 'data.frame':   607 obs. of  11 variables:
## $ work_year      : int  2020 2020 2020 2020 2020 2020 2020 2020 2020 2020 2020 ...
## $ experience_level : chr  "MI" "SE" "SE" "MI" ...
## $ employment_type : chr  "FT" "FT" "FT" "FT" ...
```

```
## $ job_title      : chr  "Data Scientist" "Machine Learning Scientist" "Big Data Engineer" "Produ
## $ salary         : int  70000 260000 85000 20000 150000 72000 190000 11000000 135000 125000 ...
## $ salary_currency : chr  "EUR" "USD" "GBP" "USD" ...
## $ salary_in_usd   : int  79833 260000 109024 20000 150000 72000 190000 35735 135000 125000 ...
## $ employee_residence: chr  "DE" "JP" "GB" "HN" ...
## $ remote_ratio    : int  0 0 50 0 50 100 100 50 100 50 ...
## $ company_location : chr  "DE" "JP" "GB" "HN" ...
## $ company_size     : chr  "L" "S" "M" "S" ...
```

```
y <- salaries$salary_in_usd / 1000
```

```
cat("Missing values in the whole dataset:", sum(is.na(salaries)), "\n")
```

```
## Missing values in the whole dataset: 0
```

```
cat("Non-positive salaries:", sum(salaries$salary_in_usd <= 0), "\n")
```

```
## Non-positive salaries: 0
```

```
cat("n =", length(y),
    " | mean:", round(mean(y), 2),
    " median:", round(median(y), 2),
    " sd:", round(sd(y), 2), "\n")
```

```
## n = 607 | mean: 112.3 median: 101.57 sd: 70.96
```

Interpretation. The data has no missing values and no non-positive salaries, so no imputation is needed. The mean is above the median, an early sign of right skew.

2.2 Univariate exploration of the response

```
hist(y, breaks = 30, freq = FALSE, col = "lightblue", border = "white",
     main = "Distribution of salary_in_usd", xlab = "Salary (thousands USD)")
lines(density(y), col = "red", lwd = 2)
```

Distribution of salary_in_usd



```
cat("Skewness:", round(skewness(y), 3), "\n")
```

```
## Skewness: 1.663
```

```
cat("Kurtosis:", round(kurtosis(y), 3), "\n")
```

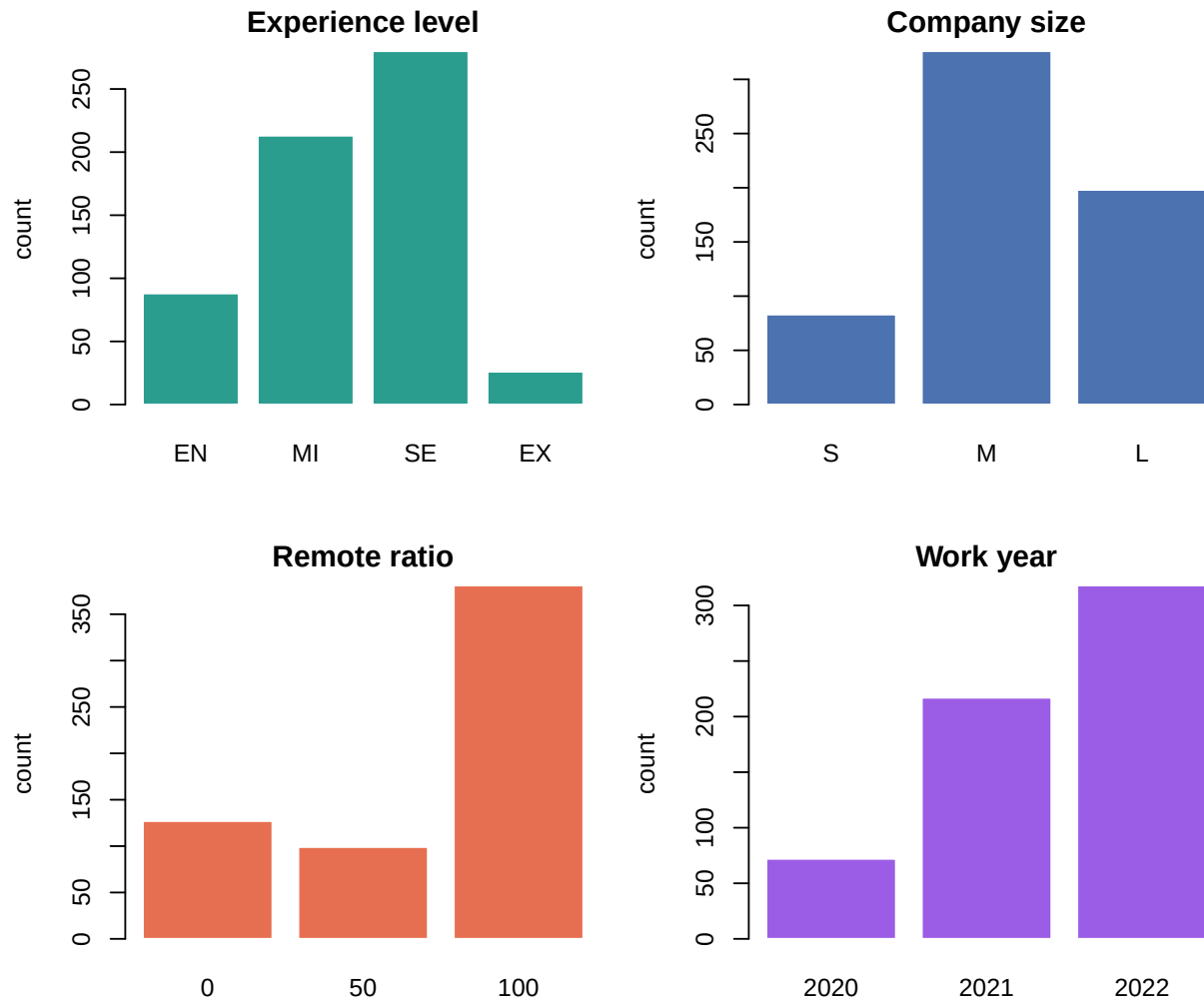
```
## Kurtosis: 9.292
```

Description of the plot. A histogram of `salary_in_usd` (light blue bars) with a smoothed density curve (red line). Most salaries sit between about 50k and 150k, with a peak near 75-100k. The bars get shorter after 200k, but a few small bumps reach out to 400-600k, showing the long right tail.

Key observations. The distribution is right-skewed with heavy tails, and a long upper tail reaches the very high salaries. On the log scale it looks close to symmetric, which supports a log link for both models and makes Log-Normal a natural competitor to Gamma.

2.3 Categorical and discrete covariate summaries

```
par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))
barplot(table(factor(salaries$experience_level, levels = c("EN", "MI", "SE", "EX"))),
        col = "#2a9d8f", border = "white", main = "Experience level", ylab = "count")
barplot(table(factor(salaries$company_size, levels = c("S", "M", "L"))),
        col = "#4c72b0", border = "white", main = "Company size", ylab = "count")
barplot(table(salaries$remote_ratio),
        col = "#e76f51", border = "white", main = "Remote ratio", ylab = "count")
barplot(table(salaries$work_year),
        col = "#9b5de5", border = "white", main = "Work year", ylab = "count")
```



```
par(mfrow = c(1, 1))
```

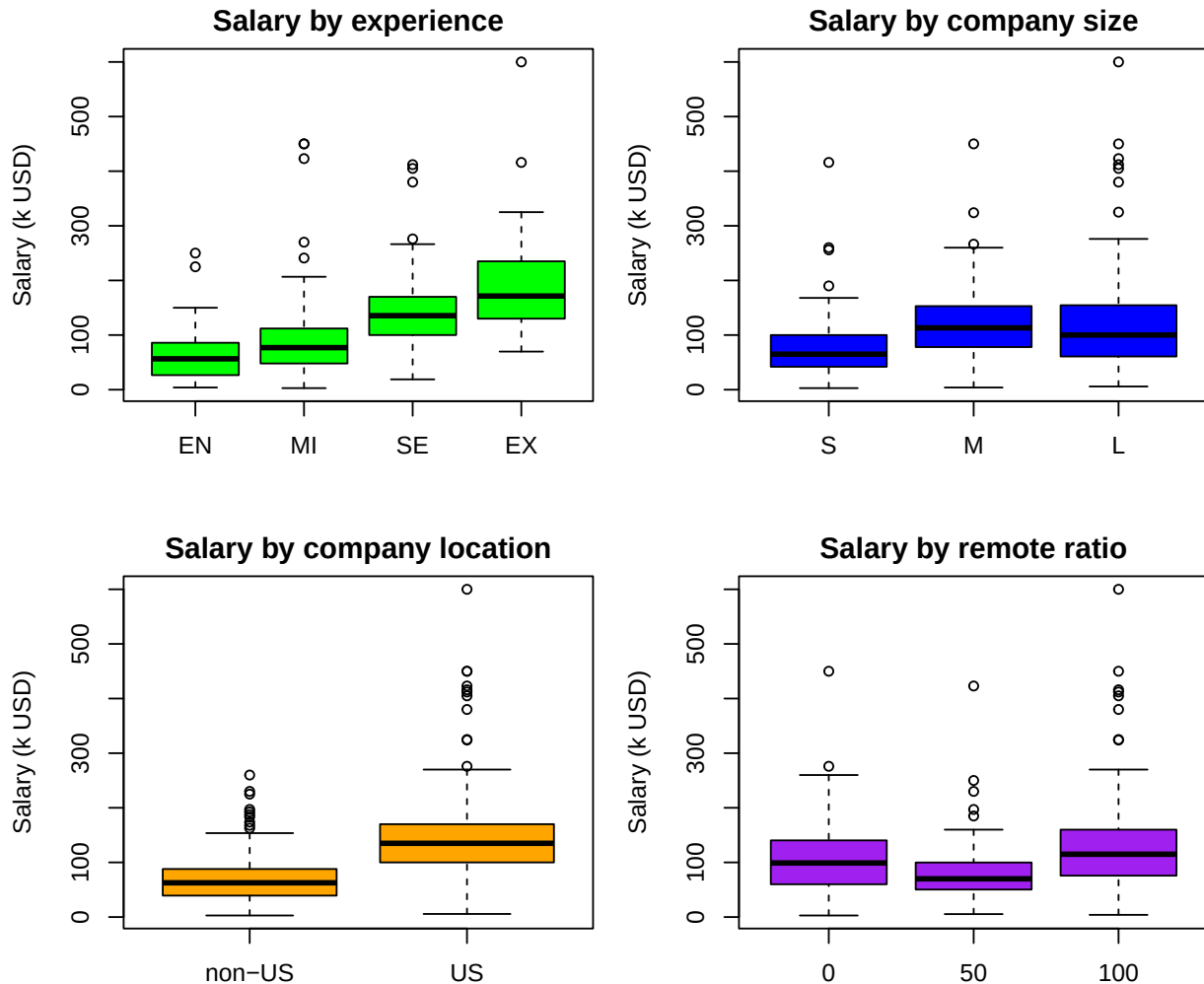
Description of the plot. Four bar charts in a 2x2 grid, one per covariate. Experience level: SE (senior) is the largest group, then MI, then EN, with EX (executive) the smallest. Company size: M (medium) is the largest group, followed by L (large), with S (small) the smallest. Remote ratio: fully remote (100) is by far the largest group, followed by fully on-site (0), with hybrid (50) the smallest. Work year: the count of records grows each year, from 2020 (smallest) to 2022 (largest).

2.4 Bivariate relationships

```
exper <- factor(salaries$experience_level, levels = c("EN", "MI", "SE", "EX"))
csize <- factor(salaries$company_size, levels = c("S", "M", "L"))
us <- factor(ifelse(salaries$company_location == "US", "US", "non-US"),
             levels = c("non-US", "US"))
remote <- factor(salaries$remote_ratio)

par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))
boxplot(y ~ exper, col = "green", main = "Salary by experience", ylab = "Salary (k USD)", xlab = "")
boxplot(y ~ csize, col = "blue", main = "Salary by company size", ylab = "Salary (k USD)", xlab = "")
```

```
boxplot(y ~ us, col = "orange", main = "Salary by company location", ylab = "Salary (k USD)", xlab = "us")
boxplot(y ~ remote, col = "purple", main = "Salary by remote ratio", ylab = "Salary (k USD)", xlab = "remote")
```



```
par(mfrow = c(1, 1))
```

Description of the plot. Four boxplots in a 2x2 grid, all on the same salary scale (0-500+ k USD) with outlier points shown as circles above each box. Salary by experience: the median box moves clearly upward from EN to MI to SE to EX. Salary by company size: S has the lowest median and a narrower box; M and L are similar and higher. Salary by company location: the US box sits much higher than the non-US box, with little overlap. Salary by remote ratio: the 0% and 100% groups have similar, higher medians, while the 50% (hybrid) group has a lower median.

Interpretation. Salary rises with experience, and US companies pay more than non-US ones. The gap between medium and small companies shrinks once experience and US location are controlled for (Section 4.2), which suggests it was mostly explained by seniority and location, not company size itself.

2.5 Response and predictor preparation

Experience level, company size and US location are kept as predictors, since Section 2.4 already showed a clear link for each. Remote ratio and work year looked weak in the boxplots, but a weak marginal link does

not rule out a real conditional one, so we test them with an F-test rather than drop them by eye. As the test below shows, the extra terms are not significant, so the final model keeps only these three predictors: experience level, company size, and US location. Remote ratio and work year are not used any further.

```
dat_test <- data.frame(
  y = y,
  exper = factor(salaries$experience_level, levels = c("EN", "MI", "SE", "EX")),
  size = factor(salaries$company_size, levels = c("S", "M", "L")),
  us = as.integer(salaries$company_location == "US"),
  remote = factor(salaries$remote_ratio),
  wyear = factor(salaries$work_year))

fit_reduced <- glm(y ~ exper + size + us,
  family = Gamma(link = "log"), data = dat_test)
fit_full <- glm(y ~ exper + size + us + remote + wyear,
  family = Gamma(link = "log"), data = dat_test)

anova_tab <- anova(fit_reduced, fit_full, test = "F")
print(anova_tab)

## Analysis of Deviance Table
##
## Model 1: y ~ exper + size + us
## Model 2: y ~ exper + size + us + remote + wyear
##   Resid. Df Resid. Dev Df Deviance      F Pr(>F)
## 1         600      165.47
## 2         596      164.73  4   0.74629 0.7278 0.5732

include_extra <- anova_tab[["Pr(>F)"]][2] < 0.05
cat("\nInclude remote_ratio and work_year, conditional on the other covariates?",
  include_extra, "\n")
```

```
##
## Include remote_ratio and work_year, conditional on the other covariates? FALSE
```

The F-test is not significant, confirming that `remote_ratio` and `work_year` add nothing once experience, size and US location are already in the model. So the design matrix X used from here on keeps only these three predictors, and `remote_ratio` and `work_year` are dropped for good.

```
dat <- data.frame(
  y = y,
  exper = factor(salaries$experience_level, levels = c("EN", "MI", "SE", "EX")),
  size = factor(salaries$company_size, levels = c("S", "M", "L")),
  us = as.integer(salaries$company_location == "US"))

X <- model.matrix(~ exper + size + us, data = dat)[, -1, drop = FALSE]
colnames(X) <- c("MI vs EN", "SE vs EN", "EX vs EN",
  "size M vs S", "size L vs S", "US vs non-US")

N <- nrow(X)
K <- ncol(X)
yv <- dat$y

cat("N =", N, " | K =", K, "covariates\n")
```

```
## N = 607 | K = 6 covariates
```

```
print(colnames(X))
```

```
## [1] "MI vs EN"      "SE vs EN"      "EX vs EN"      "size M vs S"   "size L vs S"
## [6] "US vs non-US"
```

All predictors are categorical, so coefficients are direct log-multiplicative effects relative to the reference category, and no standardisation is needed. Every JAGS model, prior, table and prediction from Section 3 onward uses `K` and `colnames(X)`.

3 Models and Inferential Goals

3.1 The two competing models

Model 1: Gamma GLM (log link), covariate-dependent shape.

$$y_i \sim \text{Gamma}(\nu_i, \nu_i/\mu_i), \quad \log \mu_i = \beta_0 + x_i^\top \beta, \quad \log \nu_i = \gamma_0 + x_i^\top \gamma,$$

so $E[y_i] = \mu_i$, and the coefficient of variation $CV_i = 1/\sqrt{\nu_i}$ can vary across employees. e^{β_j} is the multiplicative effect of covariate j on mean salary; e^{γ_j} is its effect on shape ν_i , and so (inversely) on relative dispersion.

Model 2: Log-Normal GLM (log link), covariate-dependent precision.

$$\log y_i \sim N(\eta_i, 1/\tau_i), \quad \eta_i = \beta_0 + x_i^\top \beta, \quad \log \tau_i = \delta_0 + x_i^\top \delta,$$

so e^{β_j} multiplies the median salary, and e^{δ_j} is the multiplicative effect of covariate j on precision τ_i , and so (inversely) on $\text{slog}_i = 1/\sqrt{\tau_i}$.

Both families also have a constant-dispersion version ($\nu_i \equiv \nu$, $\tau_i \equiv \tau$) as a simpler baseline, so we can answer the within-family question (Sections 4.4 and 5.4) and the matched-complexity, across-family question (Section 6).

3.2 Prior specification (elicited before seeing the data)

The priors are weakly informative and based on background knowledge, not on this dataset. Both dispersion submodels use the same structure and the same level of informativeness, so neither family has an advantage.

- Intercept $\beta_0 \sim N(3.7, sd = 0.5)$ (precision $\tau_0 = 4$), shared by both families.
- Covariate effects on the mean, $\beta_k \sim N(0, \text{precision} = 1)$, shared by both families.
- Gamma dispersion submodel: $\gamma_0 \sim N(1.33, \text{precision} = 1)$, centred on $\log(3.79)$, the constant- ν posterior mean, and $\gamma_k \sim N(0, \text{precision} = 1)$.
- Log-Normal dispersion submodel: $\delta_0 \sim N(1.11, \text{precision} = 1)$, centred on $\log(3.04)$, the constant- τ posterior mean, and $\delta_k \sim N(0, \text{precision} = 1)$, same form as γ_k .

Both dispersion submodels have exactly $K + 1$ parameters, the same prior family, the same prior precision, and an intercept centred on the matching constant-dispersion posterior mean.

```
prior <- list(m0 = 3.7, p0 = 4, pb = 1,
             a_disp = 2, b_disp = 0.5,
             g0 = 1.33, pg0 = 1, pg = 1, # Gamma dispersion submodel (gamma)
             d0 = 1.11, pd0 = 1, pd = 1) # Log-Normal dispersion submodel (delta)
```

3.3 JAGS model files

```

# --- Gamma, covariate-dependent shape ---
cat("
model {
  for (i in 1:N) {
    y[i] ~ dgamma(nu[i], rate[i])
    rate[i] <- nu[i] / mu[i]
    log(mu[i]) <- beta0 + inprod(X[i, ], beta[])
    log(nu[i]) <- gamma0 + inprod(X[i, ], gamma[])
    cv[i] <- 1 / sqrt(nu[i])
  }
  beta0 ~ dnorm(m0, p0)
  gamma0 ~ dnorm(g0, pg0)
  for (k in 1:K) {
    beta[k] ~ dnorm(0, pb)
    gamma[k] ~ dnorm(0, pg)
  }
}
", file = "gamma_glm.txt")

# --- Gamma, constant shape (baseline) ---
cat("
model {
  for (i in 1:N) {
    y[i] ~ dgamma(nu, rate[i])
    rate[i] <- nu / mu[i]
    log(mu[i]) <- beta0 + inprod(X[i, ], beta[])
  }
  beta0 ~ dnorm(m0, p0)
  for (k in 1:K) { beta[k] ~ dnorm(0, pb) }
  nu ~ dgamma(a_disp, b_disp)
  cv <- 1 / sqrt(nu)
}
", file = "gamma_glm_const.txt")

# --- Log-Normal, covariate-dependent precision ---
cat("
model {
  for (i in 1:N) {
    y[i] ~ dlnorm(eta[i], tau[i])
    eta[i] <- beta0 + inprod(X[i, ], beta[])
    log(tau[i]) <- delta0 + inprod(X[i, ], delta[])
    sdlog[i] <- 1 / sqrt(tau[i])
  }
  beta0 ~ dnorm(m0, p0)
  delta0 ~ dnorm(d0, pd0)
  for (k in 1:K) {
    beta[k] ~ dnorm(0, pb)
    delta[k] ~ dnorm(0, pd)
  }
}
", file = "lnorm_glm.txt")

# --- Log-Normal, constant precision (baseline) ---

```

```

cat("
model {
  for (i in 1:N) {
    y[i] ~ dlnorm(eta[i], tau)
    eta[i] <- beta0 + inprod(X[i, ], beta[])
  }
  beta0 ~ dnorm(m0, p0)
  for (k in 1:K) { beta[k] ~ dnorm(0, pb) }
  tau ~ dgamma(a_disp, b_disp)
  sdlog <- 1 / sqrt(tau)
}
", file = "lnorm_glm_const.txt")

```

Both families share the same structure: a mean submodel (β_0, β) with a log link, plus a dispersion submodel with one intercept and K slopes (γ_0, γ for Gamma; δ_0, δ for Log-Normal), fitted with matching priors.

3.4 Role of the parameters and inferential goals

β_0 is the log baseline salary, and each β_j is the adjusted log-effect of a covariate on mean (Gamma) or median (Log-Normal) salary. On the dispersion side: for the Gamma, $\gamma_j > 0$ raises ν_i and so lowers relative dispersion $CV_i = 1/\sqrt{\nu_i}$. For the Log-Normal, $\delta_j > 0$ raises τ_i and so lowers the log-scale spread $sdlog_i = 1/\sqrt{\tau_i}$.

3.5 Model validation: parameter recovery on simulated data

Before fitting the models on the real salaries, we check that the JAGS code and the MCMC settings actually work as intended. The idea is simple: we generate a fake dataset from known, chosen values of the parameters (the “true” β , γ/δ , and so on), then fit the same JAGS model on this fake data and see whether the posterior “recovers” those true values. If the posterior mean lands close to the true value, and the true value falls inside the 95% HPD interval, for every parameter, then we can trust the same model code on the real dataset. We do this once for the Gamma GLM and once for the Log-Normal GLM, covering both the mean submodel and the dispersion submodel in each case.

3.5.1 Recovery for the Gamma GLM

```

set.seed(2024)
Ns <- 600; Ks <- 6
Xs <- matrix(rnorm(Ns * Ks), Ns, Ks)
b0_true <- 4.0
b_true <- c(0.30, 0.60, 1.00, 0.10, 0.25, 0.65)
g0_true <- 1.6
g_true <- c(0.10, -0.15, 0.20, 0.00, 0.05, -0.10)

mu_s <- exp(b0_true + Xs %*% b_true)
nu_s <- exp(g0_true + Xs %*% g_true)
ys <- rgamma(Ns, shape = as.vector(nu_s), rate = as.vector(mu_s) / as.vector(nu_s))

if (!file.exists("rec_g.rds")) {
  rec_g <- jags(
    data = c(list(y = ys, X = Xs, N = Ns, K = Ks), prior),
    parameters.to.save = c("beta0", "beta", "gamma0", "gamma"),
    model.file = "gamma_glm.txt",
    n.chains = 3, n.iter = 6000, n.burnin = 1000, n.thin = 5,
    progress.bar = "none")
  saveRDS(rec_g, "rec_g.rds")
}

```

```

} else {
  rec_g <- readRDS("rec_g.rds")
}
sg <- rec_g$BUGSoutput$sims.matrix

pars_rec_g <- c("beta0", paste0("beta[", 1:Ks, "]"),
               "gamma0", paste0("gamma[", 1:Ks, "]"))
rec_tab_g <- data.frame(
  parameter = pars_rec_g,
  true       = c(b0_true, b_true, g0_true, g_true),
  post_mean  = colMeans(sg[, pars_rec_g]))
hpd_g0 <- HPDinterval(as.mcmc(sg[, pars_rec_g]))
rec_tab_g$lwr <- hpd_g0[, 1]; rec_tab_g$upr <- hpd_g0[, 2]

knitr::kable(rec_tab_g, digits = 3, row.names = FALSE,
             caption = "Gamma GLM parameter recovery: mean submodel (beta) and dispersion submodel (gamma)")

```

Table 1: Gamma GLM parameter recovery: mean submodel (β) and dispersion submodel (γ)

parameter	true	post_mean	lwr	upr
beta0	4.00	3.999	3.961	4.037
beta[1]	0.30	0.288	0.249	0.324
beta[2]	0.60	0.639	0.600	0.681
beta[3]	1.00	1.037	1.001	1.072
beta[4]	0.10	0.107	0.071	0.144
beta[5]	0.25	0.264	0.225	0.302
beta[6]	0.65	0.640	0.604	0.673
gamma0	1.60	1.531	1.423	1.641
gamma[1]	0.10	0.088	-0.014	0.197
gamma[2]	-0.15	-0.149	-0.257	-0.045
gamma[3]	0.20	0.276	0.162	0.390
gamma[4]	0.00	-0.029	-0.146	0.093
gamma[5]	0.05	0.034	-0.076	0.143
gamma[6]	-0.10	-0.067	-0.173	0.051

Interpretation. Recovery is good overall: for 12 of the 14 parameters the posterior mean sits close to the true value and the 95% HPD interval covers it comfortably, for both the mean submodel (β) and the dispersion submodel (γ). Two coefficients are borderline: **beta[2]** (SE vs EN) has its true value of 0.60 right at the edge of the interval (lwr = 0.600), and **beta[3]** (EX vs EN) has its true value of 1.00 just outside the lower bound (lwr = 1.001, a gap of 0.001). Both misses are tiny and consistent with ordinary MCMC sampling noise on a single simulated dataset, not a sign of systematic bias in the model. Overall this still supports trusting the Gamma model's coefficients found on the real data in Section 4.2.

3.5.2 Recovery for the Log-Normal GLM

```

set.seed(4242)
Xs2 <- matrix(rnorm(Ns * Ks), Ns, Ks)
d0_true <- 1.2
d_true <- c(-0.10, 0.20, 0.15, 0.05, -0.05, 0.25)

eta_s <- b0_true + Xs2 %*% b_true

```

```

tau_s <- exp(d0_true + Xs2 %*% d_true)
ys2 <- rlnorm(Ns, meanlog = as.vector(eta_s), sdlog = 1 / sqrt(as.vector(tau_s)))

if (!file.exists("rec_1.rds")) {
  rec_1 <- jags(
    data = c(list(y = ys2, X = Xs2, N = Ns, K = Ks), prior),
    parameters.to.save = c("beta0", "beta", "delta0", "delta"),
    model.file = "lnorm_glm.txt",
    n.chains = 3, n.iter = 6000, n.burnin = 1000, n.thin = 5,
    progress.bar = "none")
  saveRDS(rec_1, "rec_1.rds")
} else {
  rec_1 <- readRDS("rec_1.rds")
}
sl <- rec_1$BUGSoutput$sims.matrix

pars_rec_1 <- c("beta0", paste0("beta[", 1:Ks, "]"),
               "delta0", paste0("delta[", 1:Ks, "]"))
rec_tab_1 <- data.frame(
  parameter = pars_rec_1,
  true       = c(b0_true, b_true, d0_true, d_true),
  post_mean  = colMeans(sl[, pars_rec_1])
)
hpd_10 <- HPDinterval(as.mcmc(sl[, pars_rec_1]))
rec_tab_1$lwr <- hpd_10[, 1]; rec_tab_1$upr <- hpd_10[, 2]

knitr::kable(rec_tab_1, digits = 3, row.names = FALSE,
              caption = "Log-Normal GLM parameter recovery: mean submodel (beta) and dispersion submodel (delta)")

```

Table 2: Log-Normal GLM parameter recovery: mean submodel (beta) and dispersion submodel (delta)

parameter	true	post_mean	lwr	upr
beta0	4.00	4.054	4.009	4.097
beta[1]	0.30	0.291	0.250	0.331
beta[2]	0.60	0.617	0.578	0.659
beta[3]	1.00	0.995	0.951	1.042
beta[4]	0.10	0.106	0.061	0.149
beta[5]	0.25	0.245	0.203	0.286
beta[6]	0.65	0.659	0.621	0.702
delta0	1.20	1.233	1.113	1.345
delta[1]	-0.10	-0.022	-0.139	0.083
delta[2]	0.20	0.237	0.124	0.371
delta[3]	0.15	0.192	0.079	0.318
delta[4]	0.05	-0.035	-0.151	0.087
delta[5]	-0.05	-0.055	-0.167	0.063
delta[6]	0.25	0.259	0.135	0.375

Interpretation. Recovery is good here too: 13 of the 14 parameters have their true value comfortably inside the 95% HPD interval, for both the mean submodel (β) and the dispersion submodel (δ). The one borderline case is **beta0**, the intercept: its true value of 4.00 falls just under the lower bound of the interval (lwr = 4.009, a gap of 0.009), a small miss that is typical of MCMC sampling noise rather than a systematic problem. All six **delta** coefficients, the new dispersion parameters, are recovered cleanly. Overall this supports trusting

the `delta` coefficients found on the real data in Section 5.2.

Conclusions. Both models pass the recovery check: almost every parameter, mean and dispersion alike, lands close to its true value with the 95% HPD interval covering it, and the handful of borderline misses (`beta[2]` and `beta[3]` for the Gamma, `beta0` for the Log-Normal) are all off by a hundredth of a unit or less, well within what ordinary MCMC sampling noise on a single simulated run would produce. There is no sign that either JAGS script, mean submodel or dispersion submodel, is biased. This gives confidence to move on and fit both models on the real salary data in the next two chapters.

4 Model 1: Gamma GLM

This chapter fits the Gamma model to the real salary data and reads off what it says about salaries. Section 4.1 fits two versions side by side: the covariate-dependent model, where both the mean and the dispersion ν_i depend on experience, company size and US location, and the constant-dispersion baseline, where only the mean depends on the covariates. Section 4.2 turns the posterior samples from the covariate-dependent fit into readable tables and interprets the results. Section 4.3 checks MCMC convergence, and Section 4.4 compares the two versions by DIC to see whether the extra dispersion flexibility earns its keep.

4.1 Fit on the real data

This section runs JAGS twice on the real dataset ($N = 607$, $K = 6$ covariates): once for `gamma_glm.txt`, the covariate-dependent model, and once for `gamma_glm_const.txt`, the constant-dispersion baseline. Both use 3 chains with different starting values, 20,000 iterations, a 5,000 burn-in and thinning of 15, giving 3,000 posterior draws each. The results are cached to disk (`M_gamma.rds`, `M_gamma_const.rds`) so the models are only re-fitted if those files are missing.

```
data_gamma <- c(list(y = yv, X = X, N = N, K = K), prior)
inits_gamma <- list(
  list(beta0 = 3.5, beta = rep( 0.0, K), gamma0 = 1.0, gamma = rep( 0.0, K)),
  list(beta0 = 4.0, beta = rep( 0.3, K), gamma0 = 1.3, gamma = rep( 0.1, K)),
  list(beta0 = 4.5, beta = rep(-0.3, K), gamma0 = 1.6, gamma = rep(-0.1, K))

if (!file.exists("M_gamma.rds")) {
  M_gamma <- jags(
    data = data_gamma, inits = inits_gamma,
    parameters.to.save = c("beta0", "beta", "gamma0", "gamma", "nu", "cv"),
    model.file = "gamma_glm.txt",
    n.chains = 3, n.iter = 20000, n.burnin = 5000, n.thin = 15,
    DIC = TRUE, progress.bar = "none")
  saveRDS(M_gamma, "M_gamma.rds")
} else {
  M_gamma <- readRDS("M_gamma.rds")
}

sims_g <- M_gamma$BUGSoutput$sims.matrix
pars_g <- c("beta0", paste0("beta[", 1:K, "]"))

inits_const_g <- list(
  list(beta0 = 3.5, beta = rep( 0.0, K), nu = 2),
  list(beta0 = 4.0, beta = rep( 0.3, K), nu = 5),
  list(beta0 = 4.5, beta = rep(-0.3, K), nu = 8))

if (!file.exists("M_gamma_const.rds")) {
  M_gamma_const <- jags(
    data = data_gamma, inits = inits_const_g,
    parameters.to.save = c("beta0", "beta", "nu", "cv"),
```

```

model.file = "gamma_glm_const.txt",
n.chains = 3, n.iter = 20000, n.burnin = 5000, n.thin = 15,
DIC = TRUE, progress.bar = "none")
saveRDS(M_gamma_const, "M_gamma_const.rds")
} else {
  M_gamma_const <- readRDS("M_gamma_const.rds")
}

```

4.2 Interpretation of the model

This section builds two posterior summary tables from the covariate-dependent Gamma fit: one for the mean submodel (β), and one for the dispersion submodel (γ). For each coefficient we report the posterior mean, the 95% HPD interval, the posterior probability of a positive effect (P_pos), and the multiplicative effect on the original salary scale (e^β or e^γ , via the posterior median to avoid Jensen's bias).

```

hpd_g <- HPDinterval(as.mcmc(sims_g[, pars_g]))
ctab_g <- data.frame(
  term      = c("(Intercept)", colnames(X)),
  post_mean = colMeans(sims_g[, pars_g]),
  lwr = hpd_g[, 1], upr = hpd_g[, 2],
  P_pos     = sapply(pars_g, function(p) mean(sims_g[, p] > 0)),
  mult_effect = apply(sims_g[, pars_g], 2, function(z) median(exp(z)))
knitr::kable(ctab_g, digits = 3, row.names = FALSE,
  caption = "Gamma GLM: posterior summaries, mean submodel")

```

Table 3: Gamma GLM: posterior summaries, mean submodel

term	post_mean	lwr	upr	P_pos	mult_effect
(Intercept)	3.774	3.606	3.937	1.000	43.614
MI vs EN	0.322	0.176	0.480	1.000	1.381
SE vs EN	0.585	0.450	0.729	1.000	1.797
EX vs EN	0.979	0.767	1.183	1.000	2.657
size M vs S	0.036	-0.112	0.193	0.694	1.035
size L vs S	0.197	0.035	0.353	0.991	1.218
US vs non-US	0.601	0.509	0.683	1.000	1.823

```

pars_gam <- c("gamma0", paste0("gamma[", 1:K, "]"))
hpd_gam <- HPDinterval(as.mcmc(sims_g[, pars_gam]))
ctab_gam <- data.frame(
  term      = c("(Intercept, log-nu)", colnames(X)),
  post_mean = colMeans(sims_g[, pars_gam]),
  lwr = hpd_gam[, 1], upr = hpd_gam[, 2],
  P_pos     = sapply(pars_gam, function(p) mean(sims_g[, p] > 0)),
  mult_effect = apply(sims_g[, pars_gam], 2, function(z) median(exp(z)))
knitr::kable(ctab_gam, digits = 3, row.names = FALSE,
  caption = "Gamma GLM: posterior summaries, dispersion submodel log(nu[i]) = gamma0 + X[i,]

```

Table 4: Gamma GLM: posterior summaries, dispersion submodel
 $\log(\text{nu}[i]) = \text{gamma0} + X[i,] \%*\% \text{gamma}$

term	post_mean	lwr	upr	P_pos	mult_effect
(Intercept, log-nu)	0.400	0.061	0.744	0.988	1.498
MI vs EN	0.086	-0.236	0.429	0.694	1.089
SE vs EN	0.733	0.418	1.059	1.000	2.078
EX vs EN	0.572	-0.033	1.159	0.965	1.783
size M vs S	0.556	0.226	0.856	1.000	1.742
size L vs S	0.334	-0.004	0.666	0.971	1.393
US vs non-US	0.491	0.263	0.721	1.000	1.630

Interpretation. The baseline salary (Entry level, Small company, non-US) is about **\$43,600**. Salary rises with experience: 1.38x for Mid-level, 1.80x for Senior, 2.66x for Executive, all with $P_{\text{pos}} = 1.000$. US location adds 1.82x, almost as much as Senior level. Large companies pay 1.22x small ones ($P_{\text{pos}} = 0.991$), but medium vs. small shows no clear effect.

The dispersion table works the other way: a positive $\text{gamma}[k]$ means tighter, more predictable salaries, not more spread out. Senior roles are the clearest case ($\text{mult} = 2.08$, $P_{\text{pos}} = 1.000$); medium size and US location are also clear ($P_{\text{pos}} = 1.000$). Executive and large size point the same way but are less certain. Mid-level shows no clear effect on dispersion.

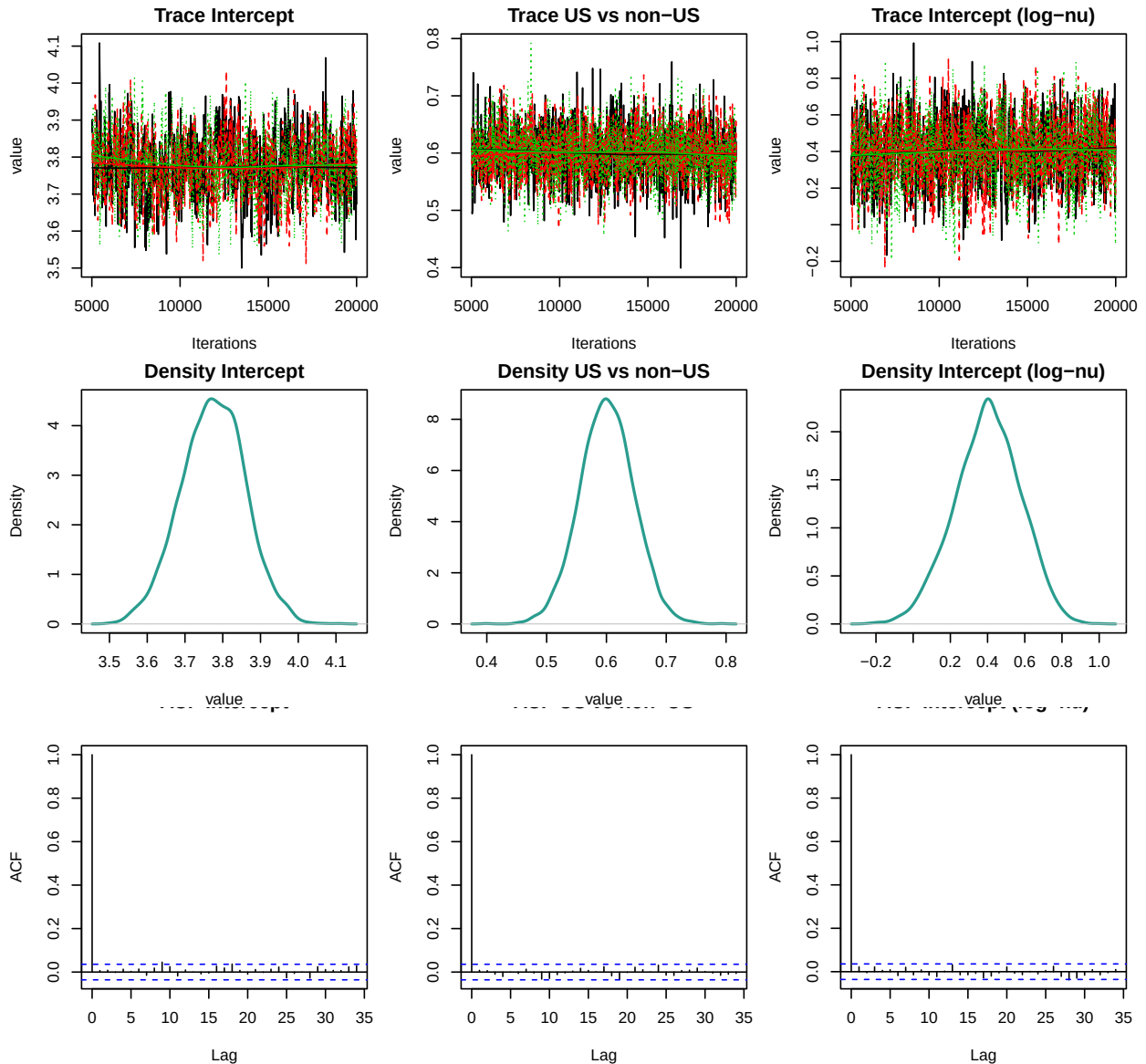
Conclusions. The same factors that raise salary, experience and US location most of all, also tend to make it more predictable. Company size matters less. Section 4.4 checks with DIC whether the extra dispersion flexibility is worth it, after Section 4.3 checks convergence.

4.3 Diagnostics

To keep this short, convergence plots are shown only for the most important parameters: the mean-submodel intercept β_0 , the strongest mean effect (US vs non-US), and the dispersion intercept γ_0 . The table below still covers every monitored parameter, so the convergence check itself is complete, only the plots are trimmed.

```
mcmc_g <- as.mcmc(M_gamma)
key_pars_g <- c("beta0", "beta[6]", "gamma0") # beta[6] = "US vs non-US"
key_labels_g <- c("Intercept", "US vs non-US", "Intercept (log-nu)")

par(mfrow = c(3, 3), mar = c(4, 4, 2, 1))
for (j in seq_along(key_pars_g)) {
  traceplot(mcmc_g[, key_pars_g[j]], col = c("black", "red", "green3"),
    main = paste("Trace", key_labels_g[j]), ylab = "value")
}
for (j in seq_along(key_pars_g)) {
  plot(density(sims_g[, key_pars_g[j]]), col = "#2a9d8f", lwd = 2,
    main = paste("Density", key_labels_g[j]), xlab = "value")
}
for (j in seq_along(key_pars_g)) {
  acf(sims_g[, key_pars_g[j]], main = paste("ACF", key_labels_g[j]))
}
```



```
par(mfrow = c(1, 1))
```

Description of the plot. A 3x3 grid for the Gamma model. Top row: traceplots for the three chains (black, red, green) of the intercept, the US vs non-US slope, and the dispersion intercept: all three look like a flat, hairy band with no trend, and the chains overlap well, a sign of good mixing. Middle row: the posterior density of each parameter, each a smooth, single-peaked (unimodal) bell-like curve. Bottom row: autocorrelation (ACF) plots: the bars drop from 1 at lag 0 to near zero almost immediately, showing the chains are not sticky.

```
gamma_pars_all <- c("beta0", paste0("beta[", 1:K, "]"),
                    "gamma0", paste0("gamma[", 1:K, "]"))
conv_g <- data.frame(
  parameter = c("Intercept", colnames(X), "Intercept (log-nu)", paste(colnames(X), "(log-nu)")),
  Rhats = gelman.diag(mcmc_g[, gamma_pars_all], multivariate = FALSE)$psrf[, 1],
  ESS = effectiveSize(mcmc_g[, gamma_pars_all]),
  MCSE = sapply(gamma_pars_all, function(p) LaplacesDemon::MCSE(sims_g[, p]))
)
knitr::kable(conv_g, digits = c(NA, 3, 0, 6), row.names = FALSE,
```

```
caption = "Convergence diagnostics, mean and dispersion submodels: Rhat, ESS, MCSE (all pa
```

Table 5: Convergence diagnostics, mean and dispersion submodels:
Rhat, ESS, MCSE (all parameters)

parameter	Rhat	ESS	MCSE
Intercept	1.001	787	0.001582
MI vs EN	1.000	1399	0.001400
SE vs EN	1.000	1254	0.001294
EX vs EN	1.000	1883	0.001890
size M vs S	1.001	1230	0.001448
size L vs S	1.001	1269	0.001451
US vs non-US	1.002	2677	0.000841
Intercept (log-nu)	1.000	1648	0.003403
MI vs EN (log-nu)	1.000	2307	0.003104
SE vs EN (log-nu)	1.000	2353	0.003024
EX vs EN (log-nu)	0.999	2778	0.005748
size M vs S (log-nu)	1.001	2094	0.003056
size L vs S (log-nu)	1.001	2737	0.003583
US vs non-US (log-nu)	1.000	3000	0.002143

Explanation. All 14 parameters converge well: $\hat{R}$ stays between 0.999 and 1.002, far below the usual 1.01 cutoff. ESS ranges from 787 (Intercept) to 3000 (US vs non-US, log-nu), so even the slowest-mixing parameter still gives hundreds of near-independent draws. MCSE is small throughout (at most 0.0057), well under the posterior spread of each parameter. So the chains can be trusted for inference.

4.4 Heteroscedastic vs constant dispersion: Gamma family

```
dic_gamma_tab <- data.frame(
  Model = c("Gamma, constant nu", "Gamma, covariate-dependent nu[i]"),
  pV = c(M_gamma_const$BUGSoutput$pV, M_gamma$BUGSoutput$pV),
  DIC = c(M_gamma_const$BUGSoutput$DIC, M_gamma$BUGSoutput$DIC))
knitr::kable(dic_gamma_tab, digits = 2,
  caption = "DIC comparison within the Gamma family: constant vs covariate-dependent dispers
```

Table 6: DIC comparison within the Gamma family: constant vs
covariate-dependent dispersion

Model	pV	DIC
Gamma, constant nu	7.90	6426.86
Gamma, covariate-dependent nu[i]	13.75	6338.89

5 Model 2: Log-Normal GLM

This chapter mirrors Chapter 4, now for the Log-Normal family. Section 5.1 fits two versions on the real data: the covariate-dependent model, where both the log-mean and the precision τ_i depend on experience, company size and US location, and the constant-dispersion baseline, where only the log-mean depends on the covariates. Section 5.2 turns the covariate-dependent posterior into readable tables and interprets the results. Section 5.3 checks MCMC convergence, and Section 5.4 compares the two versions by DIC to see whether the extra dispersion flexibility is worth it, exactly as was done for the Gamma in Section 4.4.

5.1 Fit on the real data

This section runs JAGS twice on the same real dataset used for the Gamma model ($N = 607$, $K = 6$ covariates): once for `lnorm_glm.txt`, the covariate-dependent model, and once for `lnorm_glm_const.txt`, the constant-dispersion baseline. Both use the same MCMC settings as the Gamma fits, 3 chains, 20,000 iterations, a 5,000 burn-in and thinning of 15, giving 3,000 posterior draws each, so the two families are compared on exactly equal footing. As before, results are cached to disk (`M_lnorm.rds`, `M_lnorm_const.rds`) and only re-fitted if those files are missing.

```
data_lnorm <- c(list(y = yv, X = X, N = N, K = K), prior)
inits_lnorm <- list(
  list(beta0 = 3.5, beta = rep( 0.0, K), delta0 = 0.8, delta = rep( 0.0, K)),
  list(beta0 = 4.0, beta = rep( 0.3, K), delta0 = 1.1, delta = rep( 0.1, K)),
  list(beta0 = 4.5, beta = rep(-0.3, K), delta0 = 1.4, delta = rep(-0.1, K))

if (!file.exists("M_lnorm.rds")) {
  M_lnorm <- jags(
    data = data_lnorm, inits = inits_lnorm,
    parameters.to.save = c("beta0", "beta", "delta0", "delta", "tau", "sdlog"),
    model.file = "lnorm_glm.txt",
    n.chains = 3, n.iter = 20000, n.burnin = 5000, n.thin = 15,
    DIC = TRUE, progress.bar = "none")
  saveRDS(M_lnorm, "M_lnorm.rds")
} else {
  M_lnorm <- readRDS("M_lnorm.rds")
}
sims_1 <- M_lnorm$BUGSoutput$sims.matrix
pars_1 <- c("beta0", paste0("beta[", 1:K, "]"))

inits_const_1 <- list(
  list(beta0 = 3.5, beta = rep( 0.0, K), tau = 1),
  list(beta0 = 4.0, beta = rep( 0.3, K), tau = 3),
  list(beta0 = 4.5, beta = rep(-0.3, K), tau = 6))

if (!file.exists("M_lnorm_const.rds")) {
  M_lnorm_const <- jags(
    data = data_lnorm, inits = inits_const_1,
    parameters.to.save = c("beta0", "beta", "tau", "sdlog"),
    model.file = "lnorm_glm_const.txt",
    n.chains = 3, n.iter = 20000, n.burnin = 5000, n.thin = 15,
    DIC = TRUE, progress.bar = "none")
  saveRDS(M_lnorm_const, "M_lnorm_const.rds")
} else {
  M_lnorm_const <- readRDS("M_lnorm_const.rds")
}
```

5.2 Interpretation of the model

Just as in Section 4.2, this section builds two posterior summary tables from the covariate-dependent Log-Normal fit: one for the mean submodel (β , which here targets the median salary, not the mean), and one for the dispersion submodel (δ).

```
hpd_1 <- HPDinterval(as.mcmc(sims_1[, pars_1]))
ctab_1 <- data.frame(
  term      = c("(Intercept)", colnames(X)),
  post_mean = colMeans(sims_1[, pars_1]),
```

```

lwr = hpd_1[, 1], upr = hpd_1[, 2],
P_pos = sapply(pars_1, function(p) mean(sims_1[, p] > 0)),
mult_effect = apply(sims_1[, pars_1], 2, function(z) median(exp(z)))
knitr::kable(ctab_1, digits = 3, row.names = FALSE,
caption = "Log-Normal GLM: posterior summaries, mean submodel")

```

Table 7: Log-Normal GLM: posterior summaries, mean submodel

term	post_mean	lwr	upr	P_pos	mult_effect
(Intercept)	3.489	3.280	3.680	1.000	32.776
MI vs EN	0.340	0.168	0.513	1.000	1.403
SE vs EN	0.675	0.520	0.840	1.000	1.964
EX vs EN	1.063	0.846	1.300	1.000	2.895
size M vs S	0.114	-0.057	0.282	0.902	1.121
size L vs S	0.259	0.083	0.434	0.998	1.294
US vs non-US	0.665	0.570	0.761	1.000	1.944

```

pars_del <- c("delta0", paste0("delta[", 1:K, "]"))
hpd_del <- HPDinterval(as.mcmc(sims_1[, pars_del]))
ctab_del <- data.frame(
  term = c("(Intercept, log-tau)", colnames(X)),
  post_mean = colMeans(sims_1[, pars_del]),
  lwr = hpd_del[, 1], upr = hpd_del[, 2],
  P_pos = sapply(pars_del, function(p) mean(sims_1[, p] > 0)),
  mult_effect = apply(sims_1[, pars_del], 2, function(z) median(exp(z)))
knitr::kable(ctab_del, digits = 3, row.names = FALSE,
caption = "Log-Normal GLM: posterior summaries, dispersion submodel log(tau[i]) = delta0 +

```

Table 8: Log-Normal GLM: posterior summaries, dispersion submodel $\log(\tau[i]) = \text{delta0} + X[i,] \%*\% \text{delta}$

term	post_mean	lwr	upr	P_pos	mult_effect
(Intercept, log-tau)	0.017	-0.377	0.362	0.551	1.023
MI vs EN	0.084	-0.279	0.407	0.687	1.088
SE vs EN	0.859	0.515	1.199	1.000	2.360
EX vs EN	0.761	0.139	1.394	0.988	2.169
size M vs S	0.615	0.277	0.954	1.000	1.847
size L vs S	0.479	0.113	0.825	0.998	1.611
US vs non-US	0.596	0.357	0.850	1.000	1.819

Interpretation. The baseline median salary (Entry level, Small company, non-US) is about **\$32,800**, lower than the Gamma’s mean-based \$43,600, which makes sense since the median sits below the mean for skewed data. Salary rises with experience: 1.40x for Mid-level, 1.96x for Senior, 2.90x for Executive, all with $P_{\text{pos}} = 1.000$. US location adds 1.94x. Large companies pay 1.29x small ones ($P_{\text{pos}} = 0.998$), but medium vs. small shows no clear effect, the same pattern as for the Gamma.

On dispersion, the baseline is close to zero effect ($P_{\text{pos}} = 0.551$). Almost every other covariate raises τ_i (more predictable salaries) with a fully positive HPD interval: Senior, Executive, both size categories, and US location. Only Mid-level shows no clear effect.

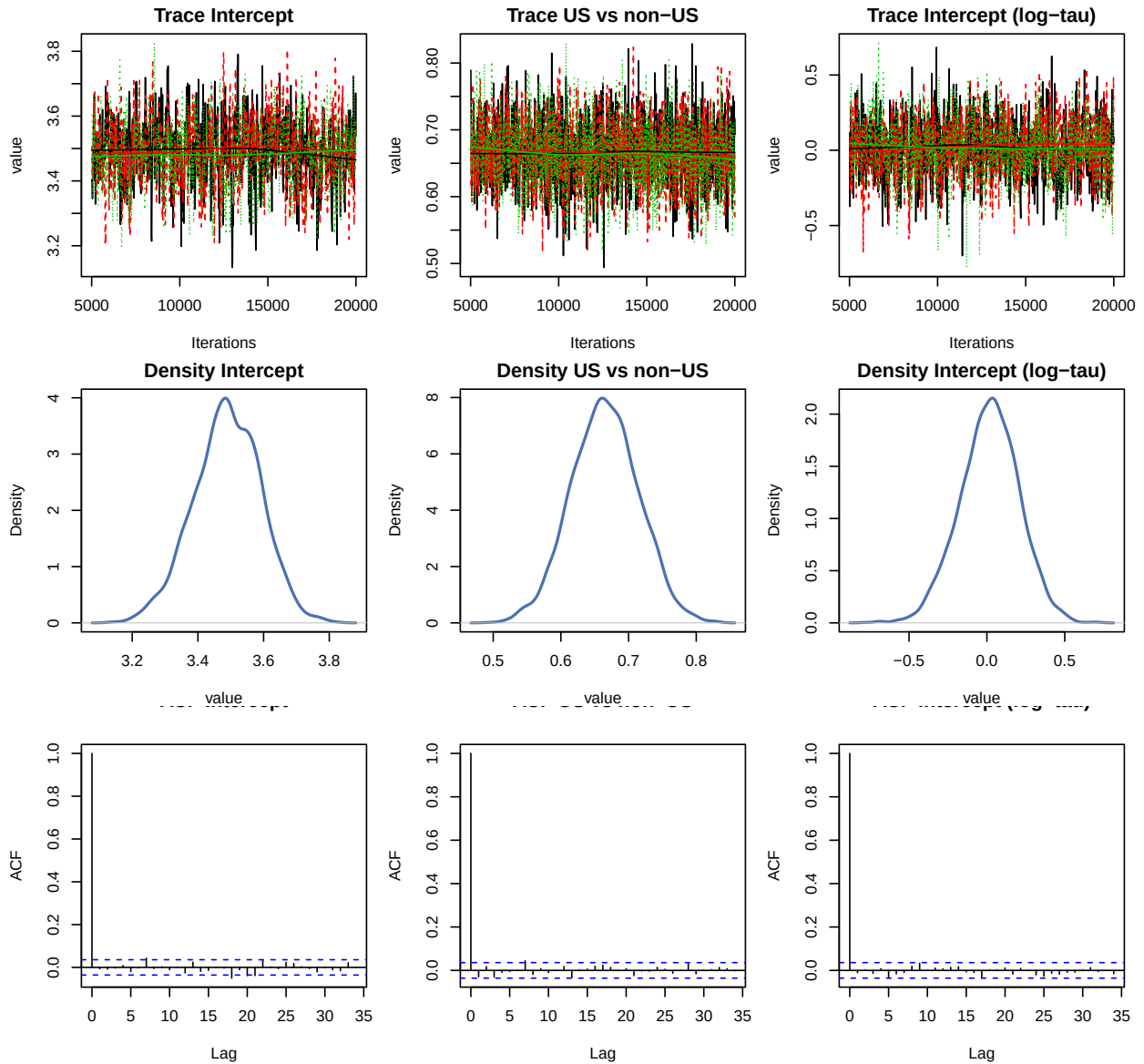
Conclusions. The story matches the Gamma model: the same covariates that raise salary also make it more predictable. The Log-Normal’s dispersion effects are a bit more decisive overall than the Gamma’s. Section 5.4 checks with DIC whether this flexibility is worth it, after Section 5.3 checks convergence.

5.3 Diagnostics

Same approach as for the Gamma: plots limited to the three most important parameters (β_0 , US vs non-US, δ_0), with the full numeric table underneath.

```
mcmc_1 <- as.mcmc(M_1norm)
key_pars_1 <- c("beta0", "beta[6]", "delta0")
key_labels_1 <- c("Intercept", "US vs non-US", "Intercept (log-tau)")

par(mfrow = c(3, 3), mar = c(4, 4, 2, 1))
for (j in seq_along(key_pars_1)) {
  traceplot(mcmc_1[, key_pars_1[j]], col = c("black", "red", "green3"),
    main = paste("Trace", key_labels_1[j]), ylab = "value")
}
for (j in seq_along(key_pars_1)) {
  plot(density(sims_1[, key_pars_1[j]]), col = "#4c72b0", lwd = 2,
    main = paste("Density", key_labels_1[j]), xlab = "value")
}
for (j in seq_along(key_pars_1)) {
  acf(sims_1[, key_pars_1[j]], main = paste("ACF", key_labels_1[j]))
}
```



```
par(mfrow = c(1, 1))
```

Description of the plot. The same 3x3 layout as for the Gamma model, now for the Log-Normal parameters (intercept, US vs non-US, dispersion intercept δ_0). The traceplots again look like flat, well-mixed bands across all three chains, the density plots are smooth and unimodal, and the ACF plots fall to near zero right after lag 0, so convergence looks just as good as for the Gamma model.

```
delta_pars_all <- c("beta0", paste0("beta[", 1:K, "]"),
                    "delta0", paste0("delta[", 1:K, "]"))
conv_l <- data.frame(
  parameter = c("Intercept", colnames(X), "Intercept (log-tau)", paste(colnames(X), "(log-tau)")),
  Rhat = gelman.diag(mcmc_l[, delta_pars_all], multivariate = FALSE)$psrf[, 1],
  ESS = effectiveSize(mcmc_l[, delta_pars_all]),
  MCSE = sapply(delta_pars_all, function(p) LaplacesDemon::MCSE(sims_l[, p]))
knitr::kable(conv_l, digits = c(NA, 3, 0, 6), row.names = FALSE,
              caption = "Convergence diagnostics, mean and dispersion submodels: Rhat, ESS, MCSE (all par
```

Table 9: Convergence diagnostics, mean and dispersion submodels: Rhat, ESS, MCSE (all parameters)

parameter	Rhat	ESS	MCSE
Intercept	1.005	688	0.001844
MI vs EN	1.003	1052	0.001626
SE vs EN	1.003	952	0.001494
EX vs EN	1.003	1470	0.002053
size M vs S	1.006	925	0.001630
size L vs S	1.005	939	0.001695
US vs non-US	1.001	2638	0.000879
Intercept (log-tau)	1.003	1334	0.003437
MI vs EN (log-tau)	1.004	2179	0.003138
SE vs EN (log-tau)	1.001	2423	0.003088
EX vs EN (log-tau)	1.000	2878	0.005621
size M vs S (log-tau)	1.001	1822	0.003271
size L vs S (log-tau)	1.001	1827	0.003545
US vs non-US (log-tau)	1.000	3368	0.002244

Explanation. Convergence is good here too, though a touch weaker than the Gamma's: $\hat{R}$ stays between 1.000 and 1.006, still well below 1.01. ESS ranges from 688 (Intercept) to 3368 (US vs non-US, log-tau); a few mean-submodel parameters (Intercept, size M/L) sit under 1000, lower than their Gamma counterparts, but still enough for reliable estimates. MCSE stays small throughout (at most 0.0056). So these chains can be trusted too.

5.4 Heteroscedastic vs constant dispersion: Log-Normal family

```
dic_lnorm_tab <- data.frame(
  Model = c("Log-Normal, constant tau", "Log-Normal, covariate-dependent tau[i]"),
  pV = c(M_lnorm_const$BUGSoutput$pV, M_lnorm$BUGSoutput$pV),
  DIC = c(M_lnorm_const$BUGSoutput$DIC, M_lnorm$BUGSoutput$DIC))
knitr::kable(dic_lnorm_tab, digits = 2,
  caption = "DIC comparison within the Log-Normal family: constant vs covariate-dependent di
```

Table 10: DIC comparison within the Log-Normal family: constant vs covariate-dependent dispersion

Model	pV	DIC
Log-Normal, constant tau	8.52	6509.27
Log-Normal, covariate-dependent tau[i]	14.07	6394.25

6 Model Comparison

Two comparisons are shown, at matched complexity on each side.

```
dic_tab_const <- data.frame(
  Model = c("Gamma, constant nu", "Log-Normal, constant tau"),
  pV = c(M_gamma_const$BUGSoutput$pV, M_lnorm_const$BUGSoutput$pV),
  DIC = c(M_gamma_const$BUGSoutput$DIC, M_lnorm_const$BUGSoutput$DIC))
knitr::kable(dic_tab_const, digits = 2,
  caption = "DIC comparison at LOW complexity (both constant dispersion)")
```

Table 11: DIC comparison at LOW complexity (both constant dispersion)

Model	pV	DIC
Gamma, constant nu	7.90	6426.86
Log-Normal, constant tau	8.52	6509.27

```
dic_tab_hetero <- data.frame(
  Model = c("Gamma, covariate-dependent nu[i]", "Log-Normal, covariate-dependent tau[i]"),
  pV = c(M_gamma$BUGSoutput$pV, M_lnorm$BUGSoutput$pV),
  DIC = c(M_gamma$BUGSoutput$DIC, M_lnorm$BUGSoutput$DIC))
knitr::kable(dic_tab_hetero, digits = 2,
  caption = "DIC comparison at HIGH complexity (both covariate-dependent dispersion)")
```

Table 12: DIC comparison at HIGH complexity (both covariate-dependent dispersion)

Model	pV	DIC
Gamma, covariate-dependent nu[i]	13.75	6338.89
Log-Normal, covariate-dependent tau[i]	14.07	6394.25

```
cat("Delta DIC, constant dispersion (Log-Normal minus Gamma):",
  round(M_lnorm_const$BUGSoutput$DIC - M_gamma_const$BUGSoutput$DIC, 1), "\n")
```

```
## Delta DIC, constant dispersion (Log-Normal minus Gamma): 82.4
```

```
cat("Delta DIC, covariate-dependent dispersion (Log-Normal minus Gamma):",
  round(M_lnorm$BUGSoutput$DIC - M_gamma$BUGSoutput$DIC, 1), "\n")
```

```
## Delta DIC, covariate-dependent dispersion (Log-Normal minus Gamma): 55.4
```

Interpretation. Both rows of both tables have a similar p_V , so each ΔDIC answers the sampling-family question cleanly at that complexity level. If Gamma wins both comparisons, the conclusion holds regardless of dispersion flexibility. If the ranking flips between the two tables, that is itself informative: it would mean the result depends on model complexity, not only on sampling family.

7 Posterior Predictive Checks

This chapter checks how well each fitted model reproduces the real data. We simulate new salaries from the posterior of each model and compare them to the observed data on four summary statistics: mean, standard deviation, skewness, and the share of salaries above 200k. We first compare both heteroscedastic models side by side, then check within each family whether letting dispersion vary by covariate actually fixes the mismatches seen with constant dispersion.

```
b0_g <- sims_g[, "beta0"]; B_g <- sims_g[, paste0("beta[", 1:K, "]")]
g0_g <- sims_g[, "gamma0"]; G_g <- sims_g[, paste0("gamma[", 1:K, "]")]
mu_g <- exp(b0_g + B_g %*% t(X))
nu_g <- exp(g0_g + G_g %*% t(X))
S <- nrow(sims_g)
yrep_g <- matrix(rgamma(S * N, shape = as.vector(nu_g),
  rate = as.vector(mu_g) / as.vector(mu_g)), nrow = S)
```

```

b0_l <- sims_l[, "beta0"]; B_l <- sims_l[, paste0("beta[", 1:K, "]")]
d0_l <- sims_l[, "delta0"]; D_l <- sims_l[, paste0("delta[", 1:K, "]")]
eta_l <- b0_l + B_l %*% t(X)
tau_l <- exp(d0_l + D_l %*% t(X))
S_l <- nrow(sims_l)
yrep_l <- matrix(rlnorm(S_l * N, meanlog = as.vector(eta_l),
                      sdlog = 1 / sqrt(as.vector(tau_l))), nrow = S_l)

Tstats <- function(v) c(mean = mean(v), sd = sd(v),
                      skew = skewness(v), prop_gt_200 = mean(v > 200))

Tobs <- Tstats(yv)
Trep_g <- t(apply(yrep_g, 1, Tstats))
Trep_l <- t(apply(yrep_l, 1, Tstats))
bayes_p <- function(Trep) colMeans(sweep(Trep, 2, Tobs, FUN = ">="))

ppc_tab <- data.frame(
  statistic = names(Tobs), observed = round(Tobs, 3),
  gamma_rep = round(colMeans(Trep_g), 3), gamma_p = round(bayes_p(Trep_g), 3),
  lnorm_rep = round(colMeans(Trep_l), 3), lnorm_p = round(bayes_p(Trep_l), 3))
knitr::kable(ppc_tab, row.names = FALSE,
             caption = "Posterior predictive checks: both families heteroscedastic")

```

Table 13: Posterior predictive checks: both families heteroscedastic

statistic	observed	gamma_rep	gamma_p	lnorm_rep	lnorm_p
mean	112.298	112.661	0.537	115.346	0.798
sd	70.957	69.310	0.298	78.188	0.896
skew	1.663	1.000	0.027	1.691	0.328
prop_gt_200	0.089	0.107	0.897	0.121	0.985

```

sims_gc <- M_gamma_const$BUGSoutput$sims.matrix
b0_gc <- sims_gc[, "beta0"]; B_gc <- sims_gc[, paste0("beta[", 1:K, "]")]
nu_gc <- sims_gc[, "nu"]
mu_gc <- exp(b0_gc + B_gc %*% t(X))
S_gc <- nrow(sims_gc)
yrep_gc <- matrix(rgamma(S_gc * N, shape = rep(nu_gc, times = N),
                      rate = rep(nu_gc, times = N) / as.vector(mu_gc)), nrow = S_gc)
Trep_gc <- t(apply(yrep_gc, 1, Tstats))

sims_lc <- M_lnorm_const$BUGSoutput$sims.matrix
b0_lc <- sims_lc[, "beta0"]; B_lc <- sims_lc[, paste0("beta[", 1:K, "]")]
sdl_lc <- sims_lc[, "sdlog"]
eta_lc <- b0_lc + B_lc %*% t(X)
S_lc <- nrow(sims_lc)
yrep_lc <- matrix(rlnorm(S_lc * N, meanlog = as.vector(eta_lc),
                      sdlog = rep(sdl_lc, times = N)), nrow = S_lc)
Trep_lc <- t(apply(yrep_lc, 1, Tstats))

ppc_within <- data.frame(
  statistic = names(Tobs), observed = round(Tobs, 3),
  gamma_const_p = round(colMeans(sweep(Trep_gc, 2, Tobs, FUN = ">=")), 3),
  gamma_hetero_p = round(bayes_p(Trep_g), 3),

```

```
lnorm_const_p = round(colMeans(sweep(Trep_lc, 2, Tobs, FUN = ">=")), 3),
lnorm_hetero_p = round(bayes_p(Trep_l), 3))
knitr::kable(ppc_within, row.names = FALSE,
             caption = "Bayesian p-values: constant vs covariate-dependent dispersion, both families")
```

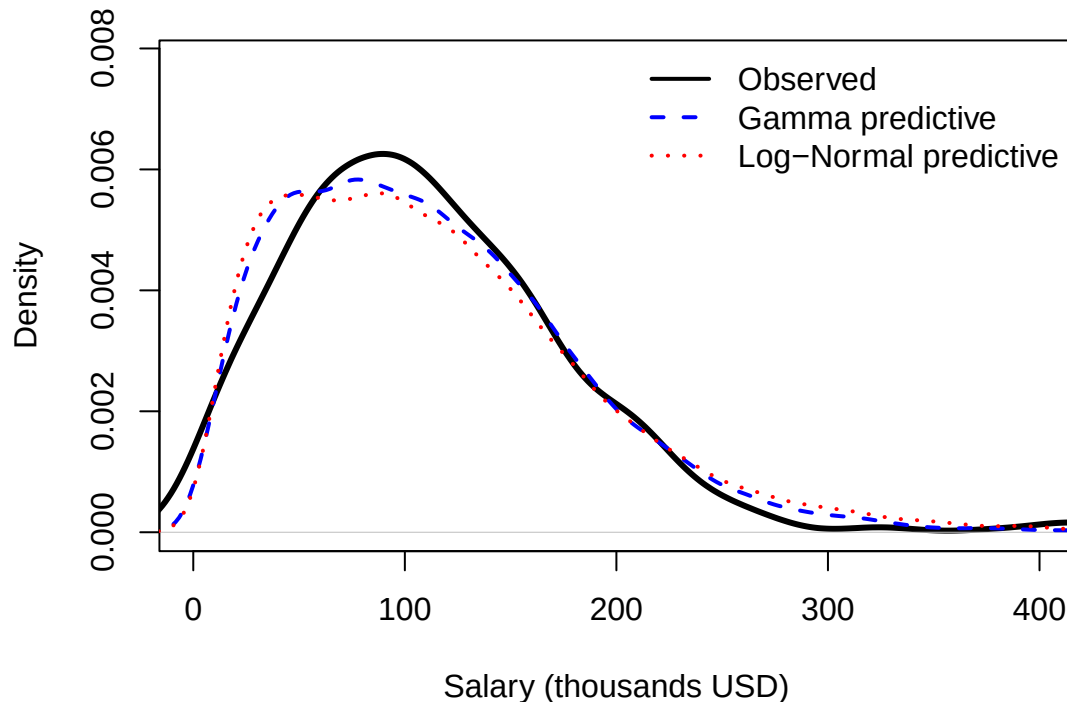
Table 14: Bayesian p-values: constant vs covariate-dependent dispersion, both families

statistic	observed	gamma_const_p	gamma_hetero_p	lnorm_const_p	lnorm_hetero_p
mean	112.298	0.532	0.537	0.931	0.798
sd	70.957	0.944	0.298	1.000	0.896
skew	1.663	0.311	0.027	0.953	0.328
prop_gt_200	0.089	0.993	0.897	1.000	0.985

Interpretation. The second table checks, within each family, whether letting dispersion vary by covariate fixed the `sd` and `prop_gt_200` mismatch seen in the constant-dispersion fits. If both `_hetero_p` columns move closer to 0.5 than their `_const_p` counterparts, the extension helps in both families.

```
dx <- density(yv)
plot(dx, col = "black", lwd = 3, ylim = c(0, max(dx$y) * 1.25),
     main = "Posterior predictive density: Gamma vs Log-Normal (both heteroscedastic)",
     xlab = "Salary (thousands USD)", xlim = c(0, 400))
lines(density(as.vector(yrep_g[sample(S, 50), ])), col = "blue", lwd = 2, lty = 2)
lines(density(as.vector(yrep_l[sample(S_l, 50), ])), col = "red", lwd = 2, lty = 3)
legend("topright", c("Observed", "Gamma predictive", "Log-Normal predictive"),
     col = c("black", "blue", "red"), lwd = 2, lty = c(1, 2, 3), bty = "n")
```

Posterior predictive density: Gamma vs Log-Normal (both heteroscedastic)



Description of the plot. Three density curves over the 0-400k salary range: the observed data (solid black line), a sample of Gamma predictive draws (dashed blue line), and a sample of Log-Normal predictive draws (dotted red line). All three curves peak around 80-100k and share the same right-skewed shape. The blue and red curves both track the black curve fairly closely, with only small differences near the peak and in the tail.

8 Inference and Prediction

This chapter uses the fitted posteriors to predict salary for three example employee profiles: an executive, a senior employee, and an entry-level employee, each with a different company size and location. For every profile we report the predicted mean (Gamma) or median (Log-Normal) salary with its own uncertainty, together with a 95% prediction interval for a single new employee with that profile.

8.1 Prediction for new employee profiles

```
make_profile <- function(...) {  
  z <- setNames(rep(0, K), colnames(X))  
  set_vals <- list(...)  
  for (nm in names(set_vals)) if (nm %in% names(z)) z[nm] <- set_vals[[nm]]  
  z  
}  
profiles <- rbind(  
  "Executive, large, US" = make_profile("EX vs EN" = 1, "size L vs S" = 1, "US vs non-US" = 1),  
  "Senior, large, US" = make_profile("SE vs EN" = 1, "size L vs S" = 1, "US vs non-US" = 1),  
  "Entry, small, non-US" = make_profile()  
)
```

```

pred_gamma <- lapply(seq_len(nrow(profiles)), function(r) {
  z <- profiles[r, ]
  mu_d <- exp(b0_g + as.numeric(B_g %*% z))
  nu_d <- exp(g0_g + as.numeric(G_g %*% z))
  ind_d <- rgamma(length(mu_d), shape = nu_d, rate = nu_d / mu_d)
  c(mean_mu = mean(mu_d), mean_nu = mean(nu_d),
    mu_HPD = HPDinterval(as.mcmc(mu_d)),
    pred_2.5 = quantile(ind_d, 0.025), pred_97.5 = quantile(ind_d, 0.975))
})
pred_tab_gamma <- as.data.frame(do.call(rbind, pred_gamma))
rownames(pred_tab_gamma) <- rownames(profiles)
names(pred_tab_gamma) <- c("E[salary] (k)", "E[nu]", "mu HPD lwr", "mu HPD upr",
  "indiv. 2.5% (k)", "indiv. 97.5% (k)")
knitr::kable(round(pred_tab_gamma, 1),
  caption = "Gamma GLM: predicted mean salary and profile-specific nu")

```

Table 15: Gamma GLM: predicted mean salary and profile-specific nu

	E[salary] (k)	E[nu]	mu HPD lwr	mu HPD upr	indiv. 2.5% (k)	indiv. 97.5% (k)
Executive, large, US	258.4	6.3	215.5	303.3	89.5	533.9
Senior, large, US	173.8	7.1	159.8	187.1	68.7	330.5
Entry, small, non-US	43.7	1.5	36.7	51.1	3.0	140.8

```

pred_lnorm <- lapply(seq_len(nrow(profiles)), function(r) {
  z <- profiles[r, ]
  eta_d <- b0_l + as.numeric(B_l %*% z)
  tau_d <- exp(d0_l + as.numeric(D_l %*% z))
  med_d <- exp(eta_d)
  ind_d <- rlnorm(length(eta_d), meanlog = eta_d, sdlog = 1 / sqrt(tau_d))
  c(median_salary = mean(med_d), mean_tau = mean(tau_d),
    med_HPD = HPDinterval(as.mcmc(med_d)),
    pred_2.5 = quantile(ind_d, 0.025), pred_97.5 = quantile(ind_d, 0.975))
})
pred_tab_lnorm <- as.data.frame(do.call(rbind, pred_lnorm))
rownames(pred_tab_lnorm) <- rownames(profiles)
names(pred_tab_lnorm) <- c("Median[salary] (k)", "E[tau]", "med HPD lwr", "med HPD upr",
  "indiv. 2.5% (k)", "indiv. 97.5% (k)")
knitr::kable(round(pred_tab_lnorm, 1),
  caption = "Log-Normal GLM: predicted median salary and profile-specific tau")

```

Table 16: Log-Normal GLM: predicted median salary and profile-specific tau

	Median[salary] (k)	E[tau]	med HPD lwr	med HPD upr	indiv. 2.5% (k)	indiv. 97.5% (k)
Executive, large, US	240.0	6.7	197.1	284.4	104.6	563.6
Senior, large, US	162.3	7.1	149.2	176.3	74.2	341.1

	Median[salary] (k)	E[tau]	med HPD lwr	med HPD upr	indiv. 2.5% (k)	indiv. 97.5% (k)
Entry, small, non-US	32.9	1.0	26.4	39.4	4.7	248.5

Interpretation. E[tau] is the Log-Normal version of the Gamma’s E[nu]: a lower E[tau] for a profile means wider log-scale spread around that profile’s own median. Comparing the two columns across the same three profiles shows whether the two families agree on which profiles carry more relative uncertainty.

9 Comparative Analysis: Frequentist vs. Bayesian

This chapter compares the Bayesian posterior means for the mean submodel with the matching frequentist estimates from `glm()` and `lm()`. This is an extra check, not part of the core Bayesian analysis, and it only covers the mean submodel, since a standard frequentist fit does not have a covariate-dependent dispersion part to compare against.

```
dat_fit <- transform(dat, salary_k = y)
fit_freq_gamma <- glm(salary_k ~ exper + size + us, family = Gamma(link = "log"), data = dat_fit)
fit_freq_lnorm <- lm(log(y) ~ exper + size + us, data = dat)

freq_vs_bayes_gamma <- data.frame(
  term = c("(Intercept)", colnames(X)),
  freq_glm = coef(fit_freq_gamma), bayes_mean = colMeans(sims_g[, pars_g]))
knitr::kable(freq_vs_bayes_gamma, digits = 3, row.names = FALSE,
  caption = "Gamma GLM: frequentist MLE vs Bayesian posterior means (mean submodel only)")
```

Table 17: Gamma GLM: frequentist MLE vs Bayesian posterior means (mean submodel only)

term	freq_glm	bayes_mean
(Intercept)	3.742	3.774
MI vs EN	0.285	0.322
SE vs EN	0.583	0.585
EX vs EN	0.995	0.979
size M vs S	0.077	0.036
size L vs S	0.203	0.197
US vs non-US	0.623	0.601

```
freq_vs_bayes_lnorm <- data.frame(
  term = c("(Intercept)", colnames(X)),
  freq_lm = coef(fit_freq_lnorm), bayes_mean = colMeans(sims_l[, pars_l]))
knitr::kable(freq_vs_bayes_lnorm, digits = 3, row.names = FALSE,
  caption = "Log-Normal GLM: frequentist MLE vs Bayesian posterior means (mean submodel only)")
```

Table 18: Log-Normal GLM: frequentist MLE vs Bayesian posterior means (mean submodel only)

term	freq_lm	bayes_mean
(Intercept)	3.427	3.489
MI vs EN	0.321	0.340

term	freq_lm	bayes_mean
SE vs EN	0.691	0.675
EX vs EN	1.112	1.063
size M vs S	0.168	0.114
size L vs S	0.283	0.259
US vs non-US	0.685	0.665

10 Conclusions and Key Findings

The Gamma GLM with covariate-dependent dispersion is the best-supported model in this analysis. At both complexity levels the Gamma beats the Log-Normal by DIC: 6426.86 vs 6509.27 with constant dispersion ($\Delta = 82.4$), and 6338.89 vs 6394.25 with covariate-dependent dispersion ($\Delta = 55.4$), a comparison that is fair since both families carry a similar p_V at each level. Letting dispersion depend on the covariates also improves both families individually, lowering the Gamma’s DIC from 6426.86 to 6338.89 and the Log-Normal’s from 6509.27 to 6394.25, so the extra flexibility earns its keep regardless of which family wins.

Experience level and US location are the strongest drivers of salary, and they matter for both its level and its spread. Relative to an entry-level, small, non-US employee, expected salary rises $\times 1.38$ for Mid-level, $\times 1.80$ for Senior, and $\times 2.66$ for Executive (Gamma estimates, all HPD intervals excluding zero), while US location alone adds $\times 1.8$ – 1.9 in both models. Company size plays a smaller role: large companies pay $\times 1.2$ – 1.3 more than small ones, but medium vs. small is not distinguishable from no effect. The same covariates that raise salary, seniority, larger company size, and US location, also raise ν_i / τ_i , meaning salaries in those groups are more predictable relative to their own mean, not just larger.

Posterior predictive checks confirm the model fits well: both heteroscedastic models reproduce the observed mean (Bayesian p-values near 0.5), and letting dispersion vary by covariate clearly improves the fit to the standard deviation and to the share of salaries above 200k compared to the constant-dispersion baselines. Skewness is the hardest feature to match, with the Gamma understating it ($p = 0.027$) more than the Log-Normal ($p = 0.328$). Predictions for example profiles are consistent with these findings and agree closely between families: an executive at a large US company is expected to earn 240k–260k, a senior employee at a large US company 160k–175k, and an entry-level employee at a small non-US company 33k–44k, with the Gamma’s prediction intervals generally narrower, especially at the low end. Finally, the Bayesian posterior means for the mean submodel are close to the matching frequentist MLEs from `glm()` and `lm()` in both families, as expected given the fairly large sample ($N = 607$) and the weakly informative priors: the data dominates the prior here.